Lecture 22: MIPS Data Types, Assembler Directives & Memory Allocation

---

🎯 Learning Objectives

By the end of this lecture, students will be able to:

- Understand the purpose of the `.data` segment.
- Learn the commonly used MIPS assembler directives.
- Declare variables of different data types.
- Understand how memory is allocated in MIPS.
- Differentiate between `.ascii` and `.asciiz`.
- Reserve memory using `.space`.
- Declare arrays and strings.
- View allocated memory inside the MARS simulator.

---

1. Introduction

🎤 Teaching Script

First 30-Second SEO Phrase

Naturally mention:

> **"In today's Computer Organization and Assembly Language lecture, we'll learn how to declare variables, strings, and arrays using MIPS assembler directives such as `.word`, `.byte`, `.half`, `.space`, `.ascii`, and `.asciiz`, and understand how memory is allocated in the MARS simulator."**

In the previous lecture, we installed the MARS simulator and wrote our very first MIPS Assembly program.

However, our program only printed a fixed message.

Real programs need variables, arrays, numbers, and strings.

So, where does the CPU store all this data?

The answer is the **Data Segment**.

In today's lecture, we'll learn how to declare variables, reserve memory, store different data types, and understand the assembler directives used to organize data in MIPS.

By the end of this lecture, you'll be able to create variables and arrays that we'll use in all upcoming Assembly programs.

Let's begin.

---

2. What is the Data Segment?

The **Data Segment** is the area of memory where a program stores:

- Variables
- Constants
- Strings
- Arrays
- Buffers

It begins with the directive:

.data

Everything written after `.data` is treated as program data until another section begins.

---

3. What are Assembler Directives?

Assembler directives are **instructions for the assembler**, not the CPU.

They tell the assembler how to:

- Allocate memory
- Store values
- Organize data
- Align memory

They **do not generate executable machine instructions**.

Examples:

.data

.text

.word

.byte

.space

.asciiz

.align

---

4. Common MIPS Data Types

Data Type	Size
Byte	1 Byte (8 bits)
Half Word	2 Bytes (16 bits)

Word	4 Bytes (32 bits)
Float	4 Bytes
Double	8 Bytes
Character	1 Byte

---

5. `.word` Directive

Used to store **32-bit integers**.

Example:

.data

number: .word 25

Memory:

number

25

Multiple values:

.data

numbers: .word 10,20,30,40

Memory:

10

20

30

40

Each value occupies **4 bytes**.

---

6. `.half` Directive

Stores **16-bit integers**.

Example:

.data

marks: .half 95

Memory required:

2 Bytes

Useful when values fit within the 16-bit range.

---

7. `.byte` Directive

Stores **8-bit values**.

Example:

.data

age: .byte 21

Memory required:

1 Byte

Another example:

.data

values: .byte 1,2,3,4,5

---

8. `.float` Directive

Stores floating-point numbers.

Example:

.data

pi: .float 3.14

Memory:

4 Bytes

---

9. `.double` Directive

Stores double-precision floating-point values.

Example:

.data

value: .double 25.678

Memory:

8 Bytes

---

10. `.ascii` Directive

Stores a string **without** a null terminator.

Example:

.data

msg: .ascii "Hello"

Memory:

H

e

l

l

o

No `\0` is added automatically.

---

11. `.asciiz` Directive

Stores a string **with** a null terminator (`\0`).

Example:

.data

msg: .asciiz "Hello"

Memory:

H

e

l

l

o

\0

Why Use `.asciiz`?

System call **4** expects a null-terminated string.

Therefore:

.asciiz

is almost always used for printing strings.

---

12. Difference Between `.ascii` and `.asciiz`

`.ascii`	`.asciiz`
No null terminator	Adds `\0` automatically

Not suitable for syscall 4	Used for printing strings
Slightly less memory	One extra byte

---

13. `.space` Directive

Used to reserve memory without assigning values.

Example:

.data

buffer: .space 100

Meaning:

Reserve:

100 Bytes

This memory can later be filled during program execution.

Common uses:

- User input
- Character arrays
- Temporary buffers

---

14. `.align` Directive

Aligns data on memory boundaries.

Example:

.align 2

Meaning:

Align data on a **4-byte boundary**.

Benefits:

- Faster memory access
- Better processor performance
- Required for some data types

Explain that beginners rarely use `.align` manually because assemblers often align data automatically, but it is important to know.

---

15. Declaring Multiple Variables

Example:

.data

age: .word 20

marks: .word 95

grade: .byte 'A'

salary: .float 25000.5

Each label points to a different memory location.

---

16. Declaring Arrays

Example:

.data

numbers: .word 5,10,15,20,25

Memory layout:

Address

10010000 → 5

10010004 → 10

10010008 → 15

10010012 → 20

10010016 → 25

Explain that each word occupies **4 bytes**, so addresses increase by 4.

---

17. Declaring Character Arrays

Example:

.data

letters: .byte 'A','B','C','D'

Each character occupies **1 byte**.

---

18. Viewing Data in MARS

Demonstrate:

1. Assemble the program.
2. Open the **Data Segment** window.
3. Locate labels.
4. Observe memory addresses.
5. Notice how different directives occupy different amounts of memory.

This helps students connect declarations with actual memory layout.

---

19. Real-Life Analogy

Imagine a warehouse.

- `.word` = Large storage boxes (4 bytes)
- `.half` = Medium boxes (2 bytes)
- `.byte` = Small boxes (1 byte)
- `.space` = Empty shelves reserved for future items
- Labels = Shelf names

The assembler organizes the warehouse before the program begins.

---

20. Common Mistakes

Students often:

❌ Use `.ascii` when printing a string.

✔ Use `.asciiz` for strings printed with `syscall 4`.

❌ Assume `.space` initializes memory to zero.

✔ `.space` only reserves memory. Do not assume its contents unless your simulator or environment guarantees initialization.

❌ Forget the colon after a label.

Incorrect:

number .word 5

Correct:

number: .word 5

🎤 Lecture Summary

Today, we learned how MIPS programs store data in memory using the `.data` segment and assembler directives.

We explored directives such as `.word`, `.half`, `.byte`, `.float`, `.double`, `.ascii`, `.asciiz`, `.space`, and `.align`, and learned how to declare variables, strings, and arrays. We also examined how memory is organized and how to inspect the Data Segment in the MARS simulator.

In the next lecture, we'll start working with **MIPS Arithmetic Instructions**, including `add`, `addi`, `sub`, `mul`, `div`, and move beyond data declaration to performing real computations with registers.

Thank you for watching, and I'll see you in the next lecture.
